

МИНОБРНАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
**«САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ ПЕТРА ВЕЛИКОГО»**

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 «Математика и компьютерные науки»

**Отчет о выполнении лабораторных работ по дисциплине
«Методы проектирования баз данных»
по теме «Нереляционные возможности PostgreSQL»**

Выполнил студент группы 5130201/40002
Принял доцент к.т.н.

_____ Гасов А.Е.
_____ Попов С.Г.

«___» _____ 2026 г.

Санкт-Петербург
2026

Содержание

Реферат	3
Введение	4
1 Лабораторная работа №1 «Представления»	5
1.1 Задача	5
1.2 Представление	5
1.3 Простой запрос	6
1.4 Сложный запрос	7
2 Лабораторная работа №2 «Триггеры»	10
2.1 Задача	10
2.2 Таблица	10
2.3 Триггер: добавление факта кураторства выставки	11
2.4 Триггер: удаление факта кураторства выставки	12
2.5 Триггер: изменение типа выставки	13
2.6 Триггер: изменение данных сотрудника	14
2.7 Триггер: удаление выставки	15
2.8 Триггер: удаление сотрудника	17
2.9 Схема БД	17
Заключение	19
Приложение	20

Реферат

Введение

1 Лабораторная работа №1 «Представления»

1.1 Задача

Реализовать представление на основе таблиц базы данных музея, а также составить простой и сложный запрос, используя данное представление. Простой запрос - это запрос непосредственно к представлению, а в сложном запросе обязательно присутствие соединения представления с другими таблицами базы данных (при помощи JOIN).

1.2 Представление

Условие для формирования представления: Для каждой записи о сотруднике (определяется полями id-сотрудника, именем и фамилией) найти число курируемых выставок и число уникальных типов среди этих выставок.

Текст представления представлен в Листинге 1.

```
1 CREATE VIEW employee_curation_stats AS
2 SELECT
3     employees.employee_id,
4     employees.first_name,
5     employees.last_name,
6     COUNT (exhibition_curators.exhibition_id) AS curated_exhibitions_count,
7     COUNT(DISTINCT exhibitions.exhibition_type_id) AS curated_exhibition_types_count
8
9 FROM
10     employees
11
12     INNER JOIN exhibition_curators
13         ON exhibition_curators.employee_id = employees.employee_id
14     INNER JOIN exhibitions
15         ON exhibitions.exhibition_id = exhibition_curators.exhibition_id
16
17 GROUP BY
18     employees.employee_id,
19     employees.first_name,
20     employees.last_name;
```

Листинг 1: Представление employee_curation_stats

Реляционная формула представления:

$$R_{view1} = \gamma_{\text{employee_id, first_name, last_name; COUNT(distinct exhibition_id), COUNT(distinct exhibition_type_id)}} \left(\begin{array}{c} \text{employees} \bowtie_{\text{employee_id}} \\ \text{exhibition_curators} \bowtie_{\text{exhibition_id}} \\ \text{exhibitions} \end{array} \right),$$

где $\bowtie$ — это natural join

1.3 Простой запрос

Текст запроса: Найти все записи о сотрудниках, которые курировали больше 3 выставок и при этом курировали выставки более чем 1 типа.

Запрос обращается к представлению `employee_curation_stats` как к обычной таблице, без соединения с другими таблицами базы данных. Текст запроса представлен в Листинге 2.

```
1 SELECT
2     employee_curation_stats.first_name,
3     employee_curation_stats.last_name,
4     employee_curation_stats.curated_exhibitions_count,
5     employee_curation_stats.curated_exhibition_types_count
6 FROM
7     employee_curation_stats
8 WHERE
9     employee_curation_stats.curated_exhibitions_count > 3
10    AND employee_curation_stats.curated_exhibition_types_count > 1;
```

Листинг 2: Простой запрос к представлению

Реляционная формула запроса:

$$T = \sigma_{\text{curated_exhibitions_count} > 3 \wedge \text{curated_exhibition_types_count} > 1}(\text{employee_curation_stats}),$$
$$R_{view2} = \pi_{\text{first_name, last_name, curated_exhibitions_count, curated_exhibition_types_count}}(T)$$

Результат выполнения запроса (выведены первые 20 строк) представлен на Рис.1.

first_name	last_name	curated_exhibitions_count	curated_exhibition_types_count
Игорь	Петров	4	2
Антон	Андреев	4	4
Дмитрий	Григорьев	4	3
Владимир	Павлов	6	3
Дмитрий	Смирнов	5	5
Степан	Степанов	8	5
Дмитрий	Егоров	4	3
Дмитрий	Лебедев	4	3
Илья	Зайцев	5	3
Иван	Захаров	6	4
Павел	Михайлов	5	4
Дмитрий	Павлов	7	4
Виктор	Волков	6	4
Максим	Яковлев	7	5
Андрей	Семенов	6	5
Николай	Васильев	5	2
Александр	Федоров	7	4
Степан	Алексеев	4	3
Сергей	Никитин	8	5
Денис	Павлов	6	4

Рис. 1: Результат выполнения простого запроса к представлению (выведены первые 20 строк)

План выполнения запроса представлен на Рис.2.

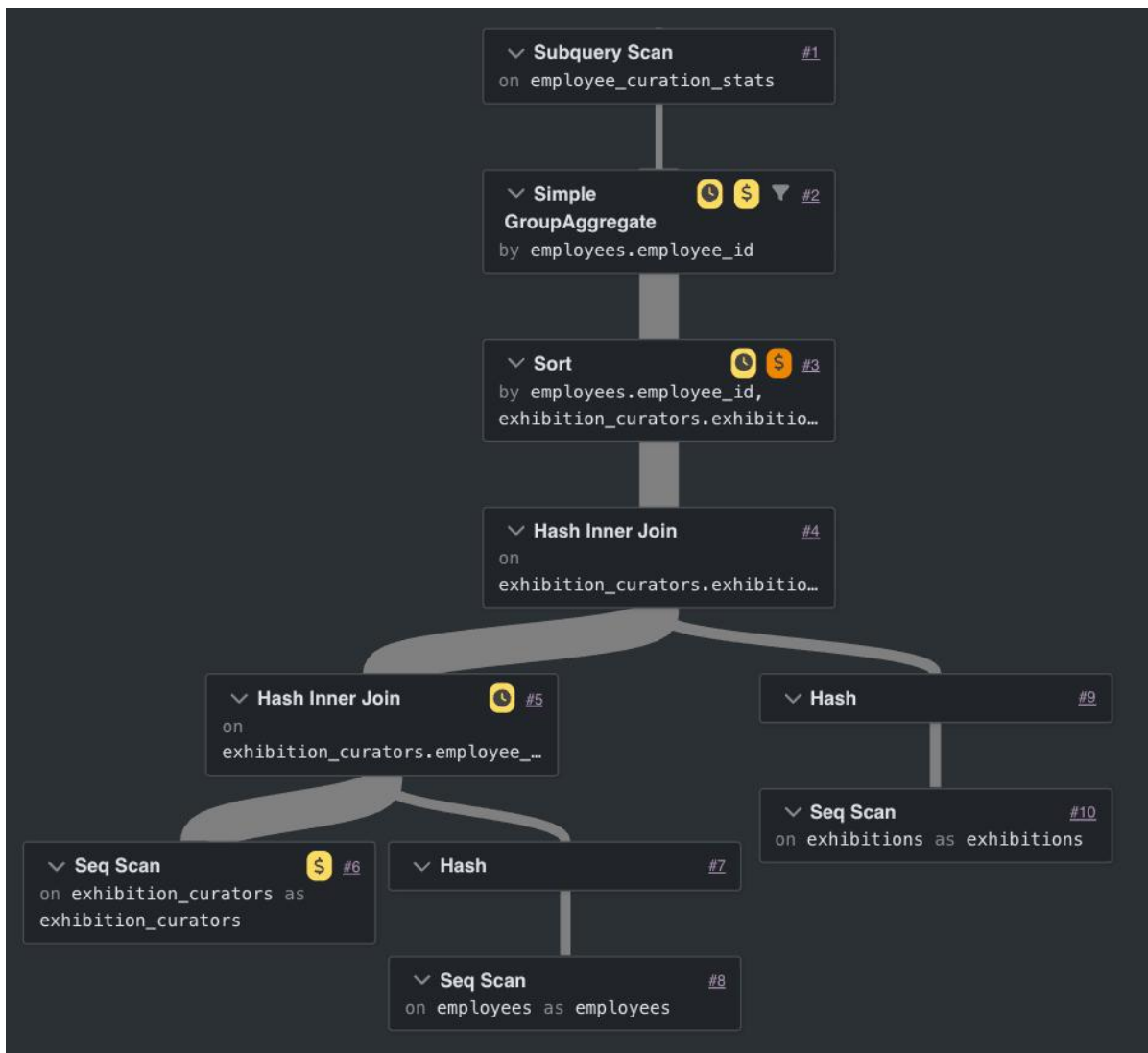


Рис. 2: План выполнения простого запроса к представлению

План разворачивает представление в подзапрос: таблицы `exhibition_curators`, `employees` и `exhibitions` последовательно соединяются через `Hash Inner Join`, результат сортируется и группируется узлом `Simple GroupAggregate`. Условие запроса (`curated_exhibitions_count > 3` и `curated_exhibition_types_count > 1`) планировщик применяет прямо внутри этого узла как фильтр после группировки, поэтому верхний узел `Subquery Scan` лишь передаёт уже отфильтрованные строки.

1.4 Сложный запрос

Текст запроса: Найти записи о технических паспортах, которых составили сотрудники с 2 и более курируемыми выставками.

Запрос соединяет (через JOIN) представление `employee_curation_stats` с таблицей `passports`. Текст запроса представлен в Листинге 3.

```

1 SELECT
2     passports.passport_id,
3     employee_curation_stats.first_name,
4     employee_curation_stats.last_name,
5     employee_curation_stats.curated_exhibitions_count
6 FROM
7     passports
8
9     INNER JOIN employee_curation_stats
10         ON employee_curation_stats.employee_id = passports.employee_id
11
12 WHERE
13     employee_curation_stats.curated_exhibitions_count >= 2;
```

Листинг 3: Сложный запрос с использованием представления

Реляционная формула запроса:

$$T = \sigma_{\text{curated_exhibitions_count} \geq 2} (\text{passports} \bowtie_{\text{employee_id}} \text{employee_curation_stats}),$$

$$R_{\text{view3}} = \pi_{\substack{\text{passport_id}, \text{first_name}, \\ \text{last_name}, \text{curated_exhibitions_count}}} (T),$$

где $\bowtie$ — это natural join

Результат выполнения запроса (выведены первые 20 строк) представлен на Рис.3.

passport_id	first_name	last_name	curated_exhibitions_count
1	Степан	Попов	3
2	Денис	Яковлев	6
3	Артем	Морозов	2
4	Игорь	Борисов	5
5	Павел	Михайлов	5
6	Денис	Иванов	4
7	Александр	Николаев	3
8	Иван	Семенов	6
9	Александр	Николаев	3
10	Владимир	Андреев	3
11	Степан	Новиков	5
12	Сергей	Яковлев	5
13	Илья	Степанов	5
14	Дмитрий	Григорьев	4
15	Денис	Семенов	3
16	Сергей	Степанов	2
17	Максим	Зайцев	3
18	Михаил	Семенов	2
19	Александр	Волков	6
21	Игорь	Андреев	2

Рис. 3: Результат выполнения сложного запроса с использованием представления (выведены первые 20 строк)

План выполнения запроса представлен на Рис.4.

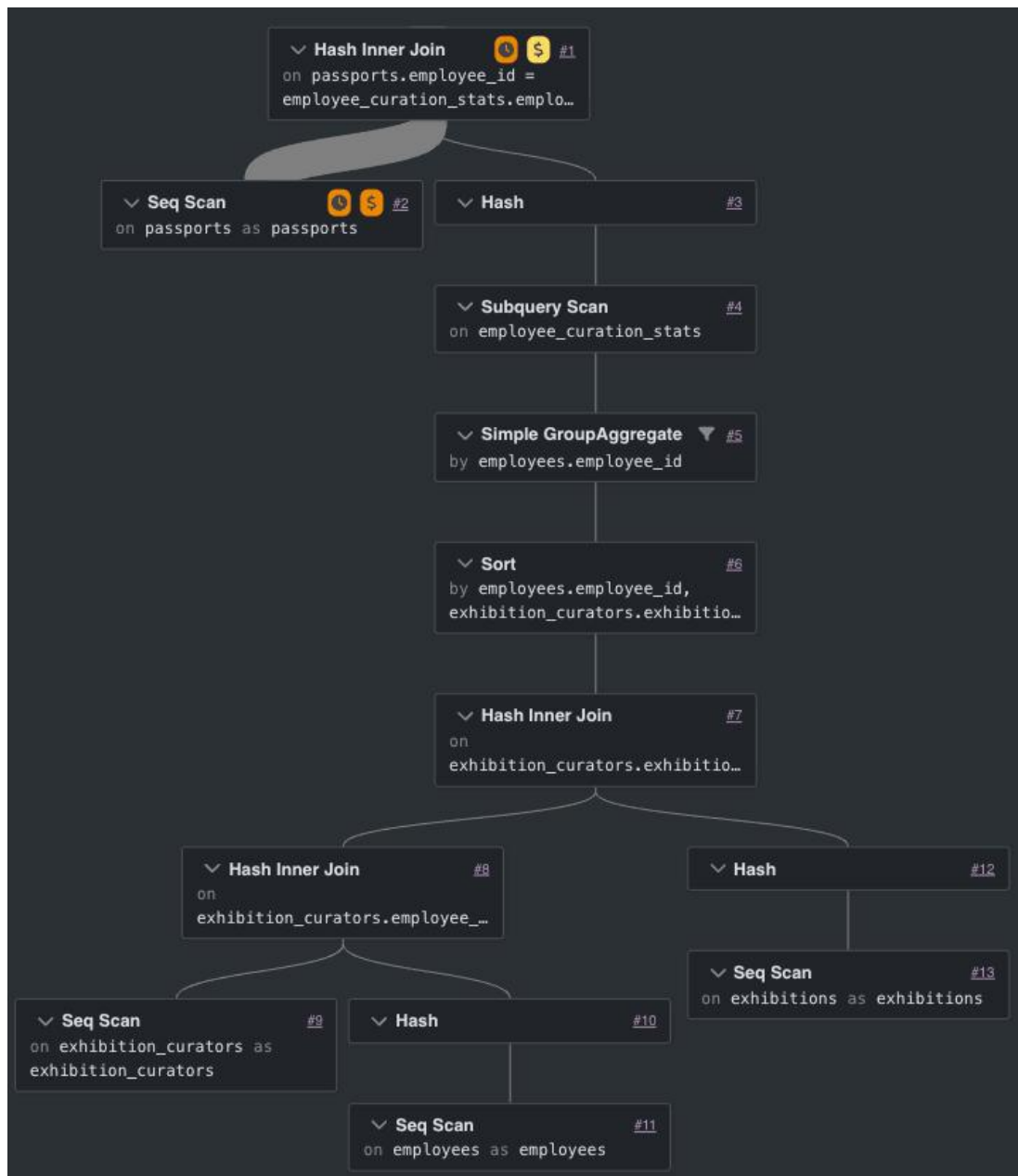


Рис. 4: План выполнения сложного запроса с использованием представления

План аналогичен предыдущему: представление снова разворачивается в тот же набор соединений, сортировку и группировку с фильтром по числу выставок (≥ 2). Сверху добавляется Hash Inner Join с таблицей **passports** — поскольку сторона представления после фильтрации заметно меньше (около 170 сотрудников против 300 тысяч паспортов), хэш-таблица строится именно по ней, а строки **passports** последовательно сканируются и зондируют эту таблицу.

2 Лабораторная работа №2 «Триггеры»

2.1 Задача

Реализовать таблицу, содержание которой повторяет представление `employee_curation_stats`. Обеспечить её актуальность с помощью триггеров: при изменении данных в таблицах `employees`, `exhibitions`, `exhibition_curators` содержимое созданной таблицы должно пересчитываться автоматически.

Должно быть реализовано 6 триггеров, покрывающих следующие изменения:

1. добавление факта кураторства выставки;
2. удаление факта кураторства выставки;
3. изменение типа выставки;
4. изменение данных сотрудника (имя, фамилия);
5. удаление выставки;
6. удаление сотрудника.

2.2 Таблица

Таблица `employee_curation_stats_table` создана на основе представления `employee_curation_stats` и содержит те же поля: идентификатор и имя сотрудника, число курируемых им выставок и число уникальных типов среди них. Текст создания таблицы представлен в Листинге 4.

```
1 CREATE TABLE employee_curation_stats_table AS
2 SELECT
3     employees.employee_id,
4     employees.first_name,
5     employees.last_name,
6     COUNT (exhibition_curators.exhibition_id) AS curated_exhibitions_count,
7     COUNT(DISTINCT exhibitions.exhibition_type_id) AS curated_exhibition_types_count
8
9 FROM
10     employees
11
12     INNER JOIN exhibition_curators
13         ON exhibition_curators.employee_id = employees.employee_id
14     INNER JOIN exhibitions
15         ON exhibitions.exhibition_id = exhibition_curators.exhibition_id
16
17 GROUP BY
18     employees.employee_id,
19     employees.first_name,
20     employees.last_name;
```

Листинг 4: Таблица `employee_curation_stats_table`

2.3 Триггер: добавление факта кураторства выставки

Срабатывает на **AFTER INSERT** по таблице **exhibition_curators**: сотруднику назначена ещё одна выставка для кураторства, его строка в **employee_curation_stats_table** пересчитывается заново. Текст триггера представлен в Листинге 5.

```
1  -- добавление факта кураторства выставки
2  CREATE OR REPLACE FUNCTION trg_curator_added()
3  RETURNS TRIGGER AS $$
4  BEGIN
5      -- удаляем старую строку из таблички, чтобы вставить обновлённую
6      DELETE FROM employee_curation_stats_table WHERE employee_id = NEW.employee_id;
7
8      INSERT INTO employee_curation_stats_table
9          (employee_id, first_name, last_name, curated_exhibitions_count,
10           curated_exhibition_types_count)
11      SELECT
12          employees.employee_id, employees.first_name, employees.last_name,
13          COUNT(exhibition_curators.exhibition_id),
14          COUNT(DISTINCT exhibitions.exhibition_type_id)
15      FROM employees
16          INNER JOIN exhibition_curators ON exhibition_curators.employee_id = employees
17          .employee_id
18          INNER JOIN exhibitions ON exhibitions.exhibition_id = exhibition_curators.
19          exhibition_id
20      WHERE employees.employee_id = NEW.employee_id
21      GROUP BY employees.employee_id, employees.first_name, employees.last_name;
22
23      RETURN NEW;
24  END;
25  $$ LANGUAGE plpgsql;
26
27  CREATE OR REPLACE TRIGGER curator_added
28      AFTER INSERT ON exhibition_curators
29      FOR EACH ROW
30      EXECUTE FUNCTION trg_curator_added();
```

Листинг 5: Триггер добавления факта кураторства

Проверка триггера (состояние **employee_curation_stats_table** до и после) представлена на Рис.5.

```

=== ТЕСТ 1: добавление факта кураторства (INSERT exhibition_curators) ===
--- Д0: сотрудник ещё ничего не курирует, строки по нему нет ---
employee_id | first_name | last_name | curated_exhibitions_count | curated_exhibition_types_count
-----+-----+-----+-----+-----
(0 rows)

INSERT 0 1
--- ПОСЛЕ: появилась строка, curated_exhibitions_count = 1, curated_exhibition_types_count = 1 ---
employee_id | first_name | last_name | curated_exhibitions_count | curated_exhibition_types_count
-----+-----+-----+-----+-----
          199 | ТестИмя   | ТестФамилия | 1 | 1
(1 row)

```

Рис. 5: Проверка триггера добавления факта кураторства

2.4 Триггер: удаление факта кураторства выставки

Срабатывает на AFTER DELETE по таблице `exhibition_curators`: у сотрудника сняли кураторство над выставкой, его строка пересчитывается; если курируемых выставок не осталось, строка сотрудника пропадает из таблицы. Текст триггера представлен в Листинге 6.

```

1  -- удаление факта кураторства выставки
2  CREATE OR REPLACE FUNCTION trg_curator_removed()
3  RETURNS TRIGGER AS $$
4  BEGIN
5      DELETE FROM employee_curation_stats_table WHERE employee_id = OLD.employee_id;
6      -- если у сотрудника не осталось курируемых выставок, INNER JOIN просто не даст
       строк
7      INSERT INTO employee_curation_stats_table
8          (employee_id, first_name, last_name, curated_exhibitions_count,
9           curated_exhibition_types_count)
10         SELECT
11             employees.employee_id, employees.first_name, employees.last_name,
12             COUNT(exhibition_curators.exhibition_id),
13             COUNT(DISTINCT exhibitions.exhibition_type_id)
14         FROM employees
15             INNER JOIN exhibition_curators ON exhibition_curators.employee_id = employees
16             .employee_id
17             INNER JOIN exhibitions ON exhibitions.exhibition_id = exhibition_curators.
18             exhibition_id
19         WHERE employees.employee_id = OLD.employee_id
20         GROUP BY employees.employee_id, employees.first_name, employees.last_name;
21     RETURN OLD;
22 END;
23 $$ LANGUAGE plpgsql;
24
25 CREATE OR REPLACE TRIGGER curator_removed
26     AFTER DELETE ON exhibition_curators
27     FOR EACH ROW
28     EXECUTE FUNCTION trg_curator_removed();

```

Листинг 6: Триггер удаления факта кураторства

Проверка триггера (состояние `employee_curation_stats_table` до и после) представлена на Рис.6.

```

=== ТЕСТ 2: удаление факта кураторства (DELETE exhibition_curators) ===
--- ДО: у сотрудника 2 курируемые выставки ---
employee_id | first_name | last_name | curated_exhibitions_count | curated_exhibition_types_count
-----
199 | ТестИмя | ТестФамилия | 2 | 1
(1 row)

DELETE 1
--- ПОСЛЕ: curated_exhibitions_count вернулся к 1 (осталась только В) ---
employee_id | first_name | last_name | curated_exhibitions_count | curated_exhibition_types_count
-----
199 | ТестИмя | ТестФамилия | 1 | 1
(1 row)

```

Рис. 6: Проверка триггера удаления факта кураторства

2.5 Триггер: изменение типа выставки

Срабатывает на `AFTER UPDATE OF exhibition_type_id` по таблице `exhibitions`: при смене типа выставки меняется число уникальных типов среди курируемых выставок у всех её кураторов, поэтому строки всех этих сотрудников пересчитываются в цикле. Текст триггера представлен в Листинге 7.

```

1  -- Триггер: изменение типа выставки -> меняется число уникальных типов
2  -- среди курируемых выставок у всех её кураторов, нужно пересчитать
3  CREATE OR REPLACE FUNCTION trg_exhibition_type_changed()
4  RETURNS TRIGGER AS $$
5  DECLARE
6      curator RECORD;
7  BEGIN
8      FOR curator IN
9          SELECT DISTINCT employee_id
10         FROM exhibition_curators
11         WHERE exhibition_id = NEW.exhibition_id
12     LOOP
13         DELETE FROM employee_curation_stats_table WHERE employee_id = curator.
employee_id;
14
15         INSERT INTO employee_curation_stats_table
16             (employee_id, first_name, last_name, curated_exhibitions_count,
17             curated_exhibition_types_count)
18         SELECT
19             employees.employee_id, employees.first_name, employees.last_name,
20             COUNT(exhibition_curators.exhibition_id),
21             COUNT(DISTINCT exhibitions.exhibition_type_id)
22         FROM employees
23             INNER JOIN exhibition_curators ON exhibition_curators.employee_id =
employees.employee_id
24             INNER JOIN exhibitions ON exhibitions.exhibition_id = exhibition_curators
.exhibition_id

```

```

24         WHERE employees.employee_id = curator.employee_id
25         GROUP BY employees.employee_id, employees.first_name, employees.last_name;
26     END LOOP;
27
28     RETURN NEW;
29 END;
30 $$ LANGUAGE plpgsql;
31
32 CREATE OR REPLACE TRIGGER exhibition_type_changed
33     AFTER UPDATE OF exhibition_type_id ON exhibitions
34     FOR EACH ROW
35     EXECUTE FUNCTION trg_exhibition_type_changed();

```

Листинг 7: Триггер изменения типа выставки

Проверка триггера (состояние `employee_curation_stats_table` до и после) представлена на Рис.7.

```

=== ТЕСТ 3: изменение типа выставки (UPDATE exhibitions.exhibition_type_id) ===
--- ДО: А и В одного типа, curated_exhibition_types_count = 1 ---

```

employee_id	first_name	last_name	curated_exhibitions_count	curated_exhibition_types_count
199	ТестИмя	ТестФамилия	2	1

```

(1 row)

UPDATE 1
--- ПОСЛЕ: у А теперь другой тип, curated_exhibition_types_count = 2 ---

```

employee_id	first_name	last_name	curated_exhibitions_count	curated_exhibition_types_count
199	ТестИмя	ТестФамилия	2	2

```

(1 row)

```

Рис. 7: Проверка триггера изменения типа выставки

2.6 Триггер: изменение данных сотрудника

Срабатывает на `AFTER UPDATE OF first_name, last_name` по таблице `employees`: при смене имени или фамилии сотрудника эти же поля синхронизируются в его строке `employee_curation_stats_table`. Счётчики не пересчитываются - смена имени на них не влияет. Текст триггера представлен в Листинге 8.

```

1  -- изменение сотрудника имяфамилия(//) -> синхронизируем в таблице
2  -- счётчики не пересчитываем - смена имени на них не влияет
3  CREATE OR REPLACE FUNCTION trg_employee_renamed()
4  RETURNS TRIGGER AS $$
5  BEGIN
6      UPDATE employee_curation_stats_table
7      SET first_name = NEW.first_name,
8          last_name = NEW.last_name
9      WHERE employee_id = NEW.employee_id;
10     RETURN NEW;
11 END;
12 $$ LANGUAGE plpgsql;

```

```

13
14 CREATE OR REPLACE TRIGGER employee_renamed
15     AFTER UPDATE OF first_name, last_name ON employees
16     FOR EACH ROW
17     EXECUTE FUNCTION trg_employee_renamed();

```

Листинг 8: Триггер изменения данных сотрудника

Проверка триггера (состояние `employee_curation_stats_table` до и после) представлена на Рис.8.

```

=== ТЕСТ 4: изменение сотрудника (UPDATE employees, смена имени) ===
--- ДО: сотрудника зовут ТестИмя ТестФамилия ---

```

employee_id	first_name	last_name	curated_exhibitions_count	curated_exhibition_types_count
199	ТестИмя	ТестФамилия	2	2

```

(1 row)

UPDATE 1
--- ПОСЛЕ: first_name синхронизировался, счётчики не изменились ---

```

employee_id	first_name	last_name	curated_exhibitions_count	curated_exhibition_types_count
199	ТестИмяПереименован	ТестФамилия	2	2

```

(1 row)

```

Рис. 8: Проверка триггера изменения данных сотрудника

2.7 Триггер: удаление выставки

Срабатывает на `BEFORE DELETE` по таблице `exhibitions`: сначала запоминаются `id` всех кураторов удаляемой выставки, затем убираются их связи в `exhibition_curators` (несмотря на `ON DELETE CASCADE`, триггер сам чистит связи заранее, т.к. он `BEFORE DELETE`, а каскад выполнится только после него), после чего строки задетых сотрудников пересчитываются одним пакетным запросом. Текст триггера представлен в Листинге 9.

```

1 -- удаление выставки -> у всех ее кураторов пропадает эта выставка из счета
2 -- поэтому пересчитываем их строки или( удаляем строку, если это была последняя
  выставка)
3 CREATE OR REPLACE FUNCTION trg_exhibition_deleted()
4 RETURNS TRIGGER AS $$
5 DECLARE
6     affected_employees INT[];
7 BEGIN
8     --запоминаем ID всех кураторов этой выставки
9     SELECT ARRAY_AGG(DISTINCT employee_id)
10     INTO affected_employees
11     FROM exhibition_curators
12     WHERE exhibition_id = OLD.exhibition_id;
13
14     --несмотря на ON DELETE CASCADE, чистим сами: триггер BEFORE DELETE, а каскад
15     --выполнится только после него - без этой строки пересчёт ниже увидит устаревшие
    связи
16     DELETE FROM exhibition_curators WHERE exhibition_id = OLD.exhibition_id;

```

```

17
18 IF affected_employees IS NOT NULL THEN
19     --удаляем старую статистику для затронутых кураторов
20     DELETE FROM employee_curation_stats_table
21     WHERE employee_id = ANY(affected_employees);
22
23     --вставляем обновленные данные
24     INSERT INTO employee_curation_stats_table
25         (employee_id, first_name, last_name, curated_exhibitions_count,
26          curated_exhibition_types_count)
27     SELECT
28         employees.employee_id,
29         employees.first_name,
30         employees.last_name,
31         COUNT(exhibition_curators.exhibition_id),
32         COUNT(DISTINCT exhibitions.exhibition_type_id)
33     FROM employees
34     INNER JOIN exhibition_curators ON exhibition_curators.employee_id =
35     employees.employee_id
36     INNER JOIN exhibitions ON exhibitions.exhibition_id = exhibition_curators
37     .exhibition_id
38     WHERE employees.employee_id = ANY(affected_employees)
39     GROUP BY employees.employee_id, employees.first_name, employees.last_name;
40 END IF;
41 RETURN OLD;
42 END;
43 $$ LANGUAGE plpgsql;
44
45 CREATE OR REPLACE TRIGGER exhibition_deleted
46 BEFORE DELETE ON exhibitions
47 FOR EACH ROW
48 EXECUTE FUNCTION trg_exhibition_deleted();

```

Листинг 9: Триггер удаления выставки

Проверка триггера (состояние `employee_curation_stats_table` до и после) представлена на Рис.9.

```

=== ТЕСТ 5: удаление выставки (DELETE exhibitions) ===
--- ДО: у сотрудника 2 курируемые выставки (А и В) ---

```

employee_id	first_name	last_name	curated_exhibitions_count	curated_exhibition_types_count
199	ТестИмяПереименован	ТестФамилия	2	2

```

(1 row)

DELETE 1
--- ПОСЛЕ: curated_exhibitions_count = 1 (осталась только В) ---

```

employee_id	first_name	last_name	curated_exhibitions_count	curated_exhibition_types_count
199	ТестИмяПереименован	ТестФамилия	1	1

```

(1 row)

```

Рис. 9: Проверка триггера удаления выставки

2.8 Триггер: удаление сотрудника

Срабатывает на AFTER DELETE по таблице `employees`: строка удалённого сотрудника убирается из `employee_curation_stats_table` (связи в `exhibition_curators` к этому моменту уже удалены каскадом). Текст триггера представлен в Листинге 10.

```
1 -- удаление сотрудника -> убираем его строку из таблицы
2 CREATE OR REPLACE FUNCTION trg_employee_deleted()
3 RETURNS TRIGGER AS $$
4 BEGIN
5     DELETE FROM employee_curation_stats_table WHERE employee_id = OLD.employee_id;
6     RETURN OLD;
7 END;
8 $$ LANGUAGE plpgsql;
9
10 CREATE OR REPLACE TRIGGER employee_deleted
11 AFTER DELETE ON employees
12 FOR EACH ROW
13 EXECUTE FUNCTION trg_employee_deleted();
```

Листинг 10: Триггер удаления сотрудника

Проверка триггера (состояние `employee_curation_stats_table` до и после) представлена на Рис.10.

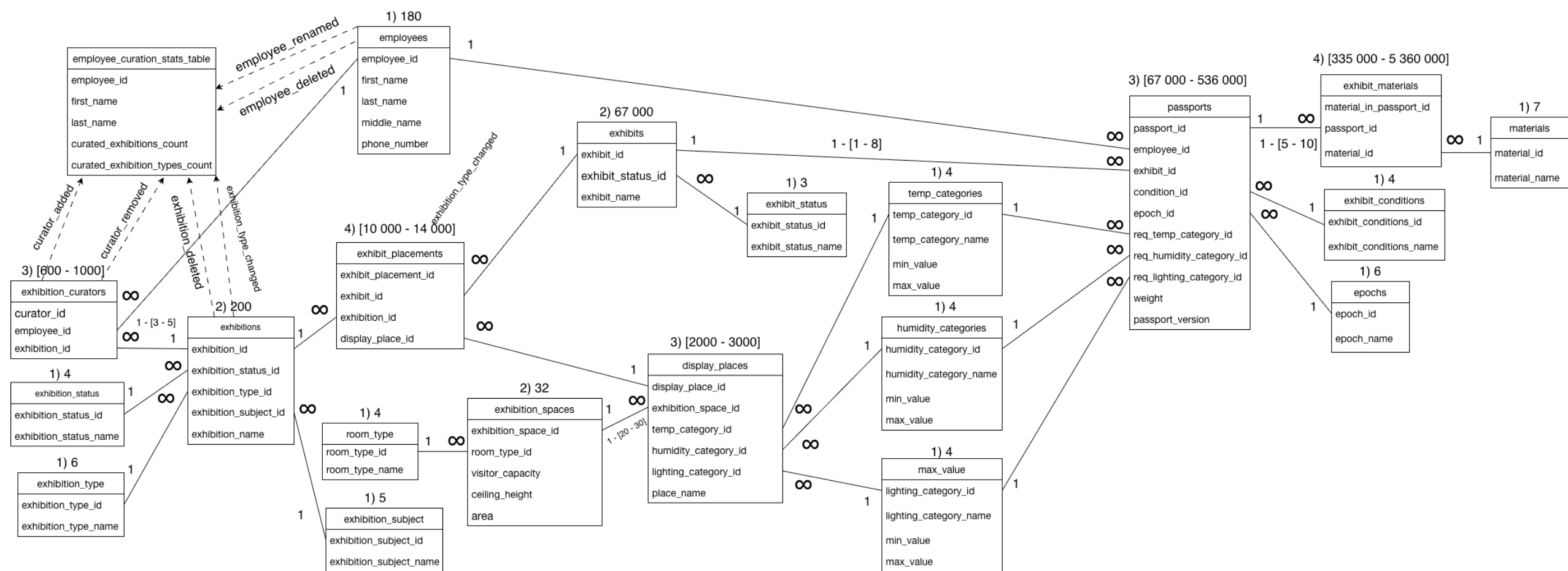
```
=== ТЕСТ 6: удаление сотрудника (DELETE employees) ===
--- ДО: у сотрудника есть курируемая выставка, строка в таблице присутствует ---
employee_id | first_name | last_name | curated_exhibitions_count | curated_exhibition_types_count
-----
209 | ТестИмяПереименован | ТестФамилия | 1 | 1
(1 row)

DELETE 1
--- ПОСЛЕ: строки по сотруднику в таблице больше нет ---
should_be_zero_table
-----
0
(1 row)
```

Рис. 10: Проверка триггера удаления сотрудника

2.9 Схема БД

Схема базы данных на английском языке с добавленной таблицей `employee_curation_stats_table` и триггерами представлена на Рис. 11.



Заключение

Приложение